Profiling MPI codes with Intel VTune

Trung Nguyen

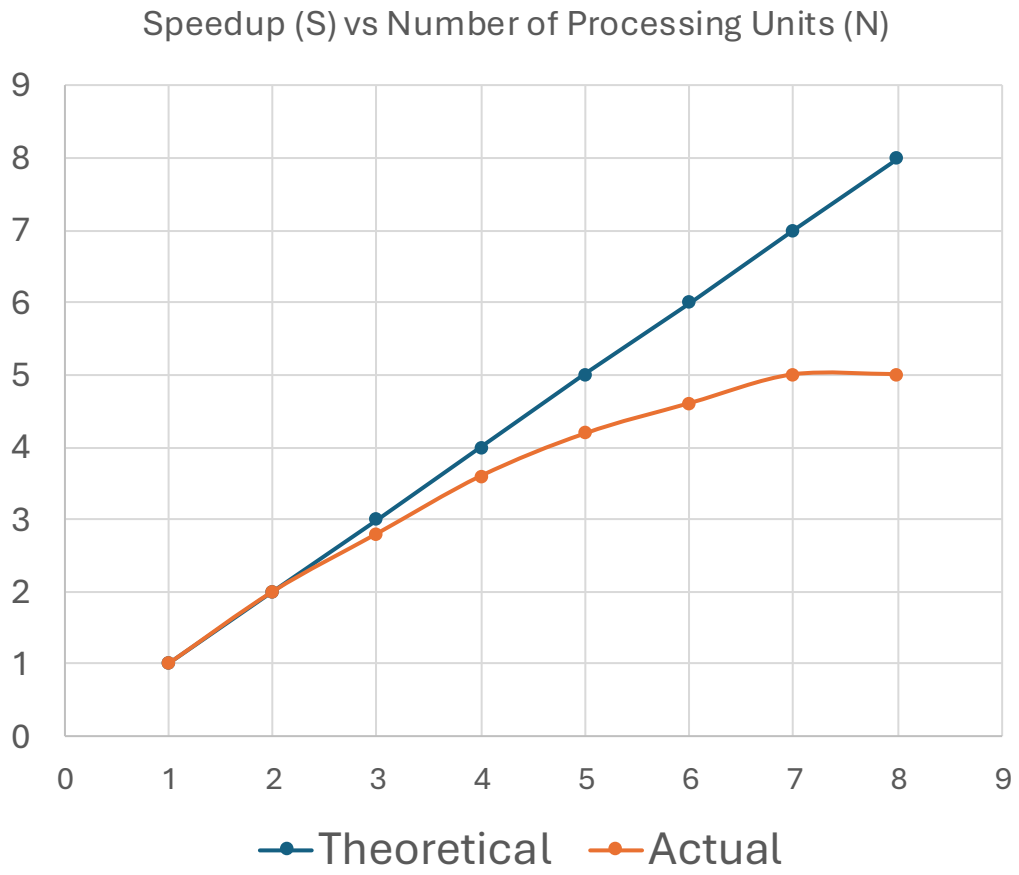
git clone https://github.com/rcc-uchicago/Midway3_profiling_knowledge_transfer.git
cd vtune

Understanding MPI codes

- MPI = Message Passing Interface
- The application built with an MPI library (e.g. OpenMPI, MPICH or Intel MPI)
- The application is launched by mpirun (or srun) in multiple processes: these processes communicate (sending and receiving data) via the dynamically linked MPI library
  - e.g. atom coords and forces between neighboring domains; pixel colors between neighboring cells
- Parallel performance: Computation gain (P/N) vs Communication overhead (O(N), O(Nlog(N)), where P = total problem size and N = number of MPI processes

Parallel efficiency: Strong scaling curve

- Parallelization introduces overhead:
  - communication/sync between workers
- Strong scaling analysis shows time to solution, or speedup, as a function of the number of MPI processes
  - linear scaling: $S_{linear} = t_1/t_N = N$
  - computation vs. communication break-even point
    - N* where speedup stops increasing with N



What to look for with profiling an MPI code?

- **High-level:** Hotspots in the source code, functions that consume most of the calculation time
  - computation within each MPI process
  - communication
- **Lower-level**:
  - across processes: load (im)balance, collective calls
  - per-process: memory access (cache hits/misses, DRAM bandwith, memory latency)

Hotspots are dependent on many factors: problem nature, solvers, parallelization scheme, and problem size

Building your MPI codes as usual

- module load mpich/4.1.2+gcc-12.2.0
- mpicxx -g -O2 –lm -o my_mpi_app my_source_code.cpp

store debugging info in the binary for function names and source code lines

or

cmake ../ -DBUILD_TYPE=RelWithDebInfo

make –j4

Running the applications with mpirun and vtune in a batch job or an interactive job

- sinteractive –-ntasks-per-node=8
- ulimit –l unlimited

Without VTune

- mpirun –np 8 your_mpi_app input.txt

With VTune

- mpirun –np 8 /software/intel/oneapi_hpc_2024.2/vtune/latest/bin64/vtune –collect hotspots –r output_dir your_mpi_app input.txt

Two examples

- mandelbrot (see vtune/README.md in the GitLab repo)
- LAMMPS: optimizing for a new pair style (atom force calculation), in collaboration with Dr. Siva Dasetty (Ferguson’s group)

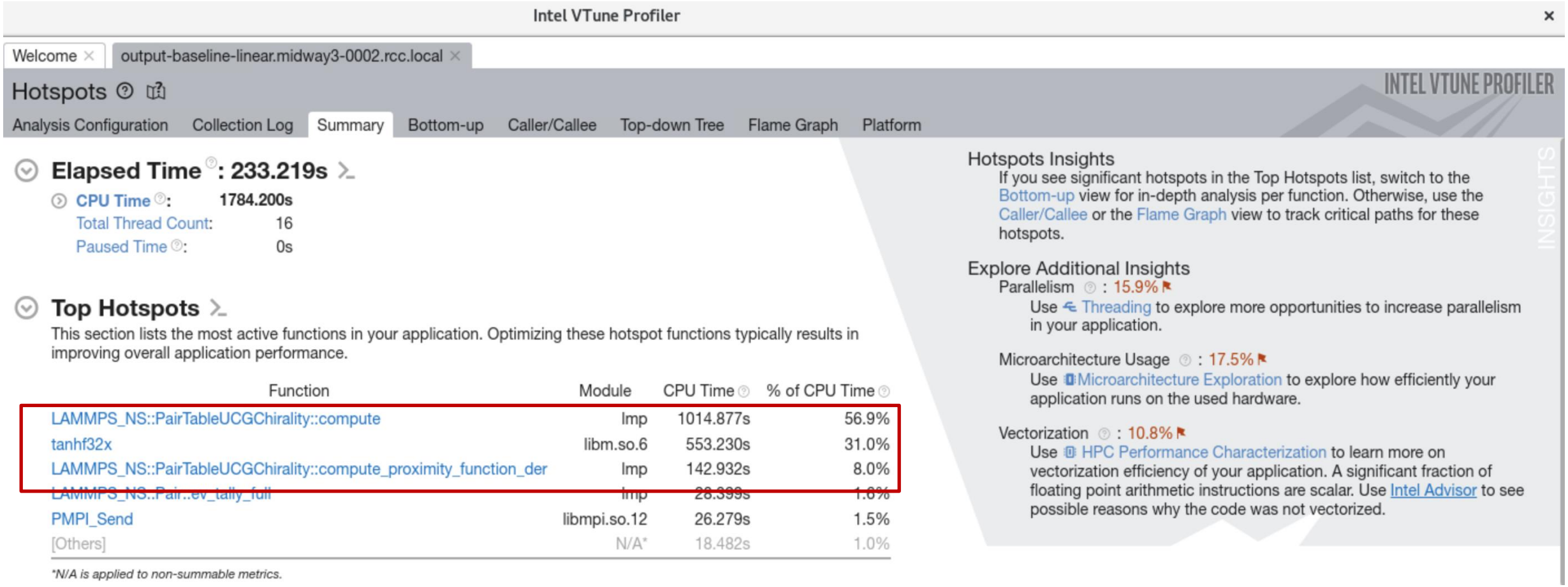
First hint: The software timing breakdown without using any profiler

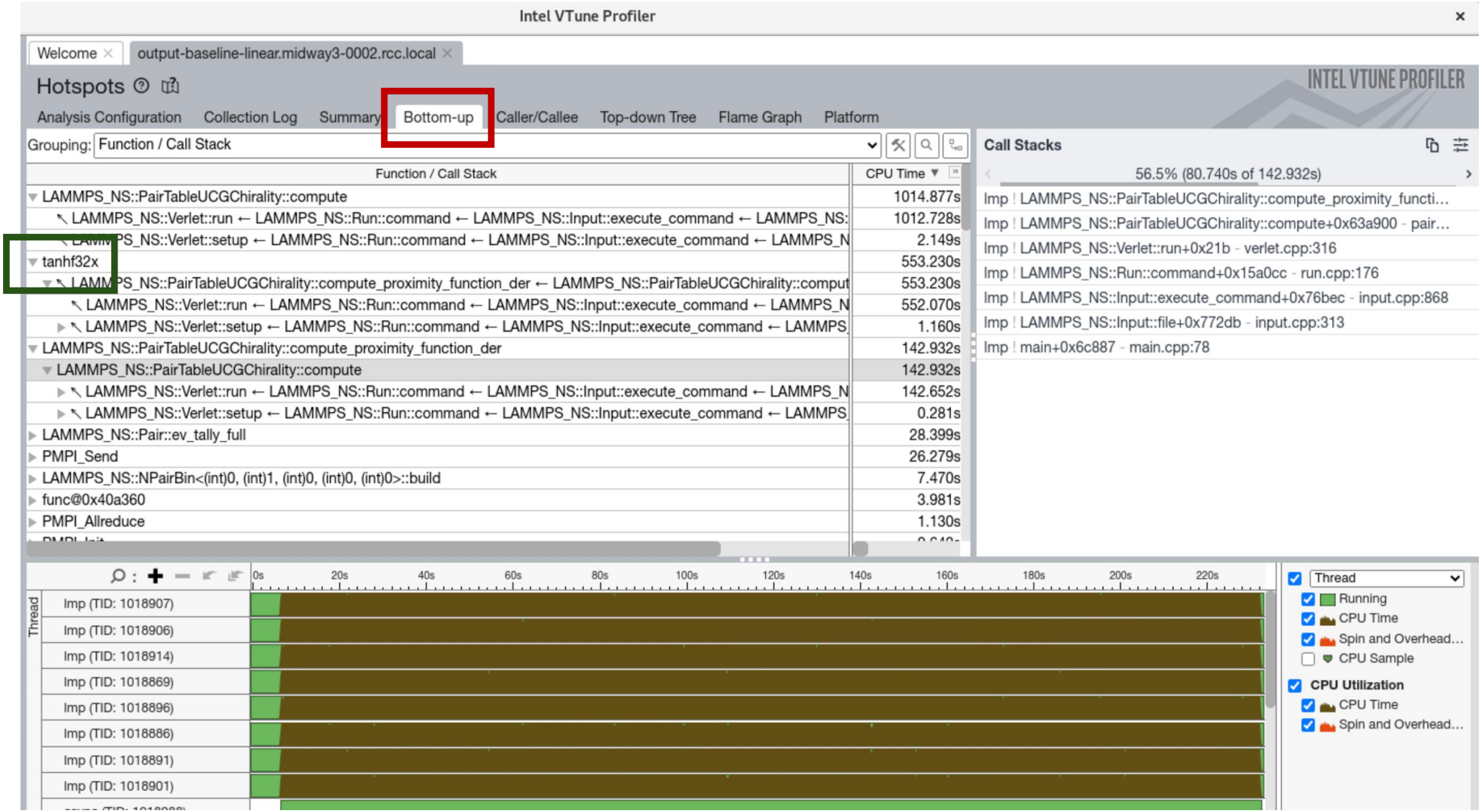
```
MPI task timing breakdown:
Section |  min time  |  avg time  |  max time  |%varavg| %total
---------------------------------------------------------------
Pair    | 223.57     | 223.57     | 223.58     |   0.0 | 99.48
Bond    | 9.83e-05   | 0.00011684 | 0.00014944 |   0.0 |  0.00
Neigh   | 0.91222    | 0.91774    | 0.92286    |   0.4 |  0.41
Comm    | 0.15603    | 0.15731    | 0.15901    |   0.2 |  0.07
Output  | 0.00045878 | 0.00046637 | 0.00049361 |   0.0 |  0.00
Modify  | 0.046506   | 0.048421   | 0.049106   |   0.4 |  0.02
Other   |            | 0.0444     |            |       |  0.02
```

Analyze the profiling report: Hotspots

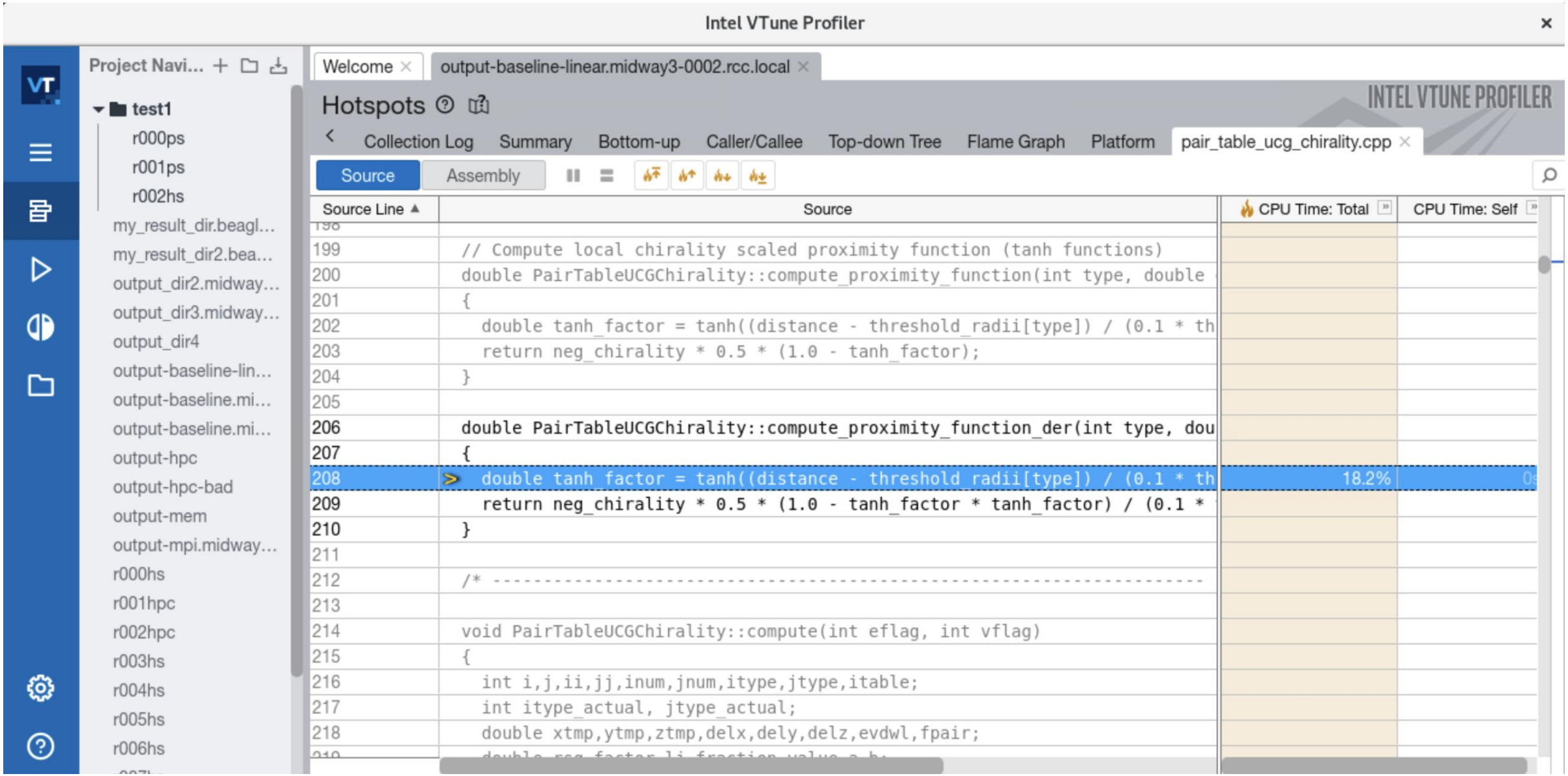
15.9% ← using 8 procs out of 48 CPU cores

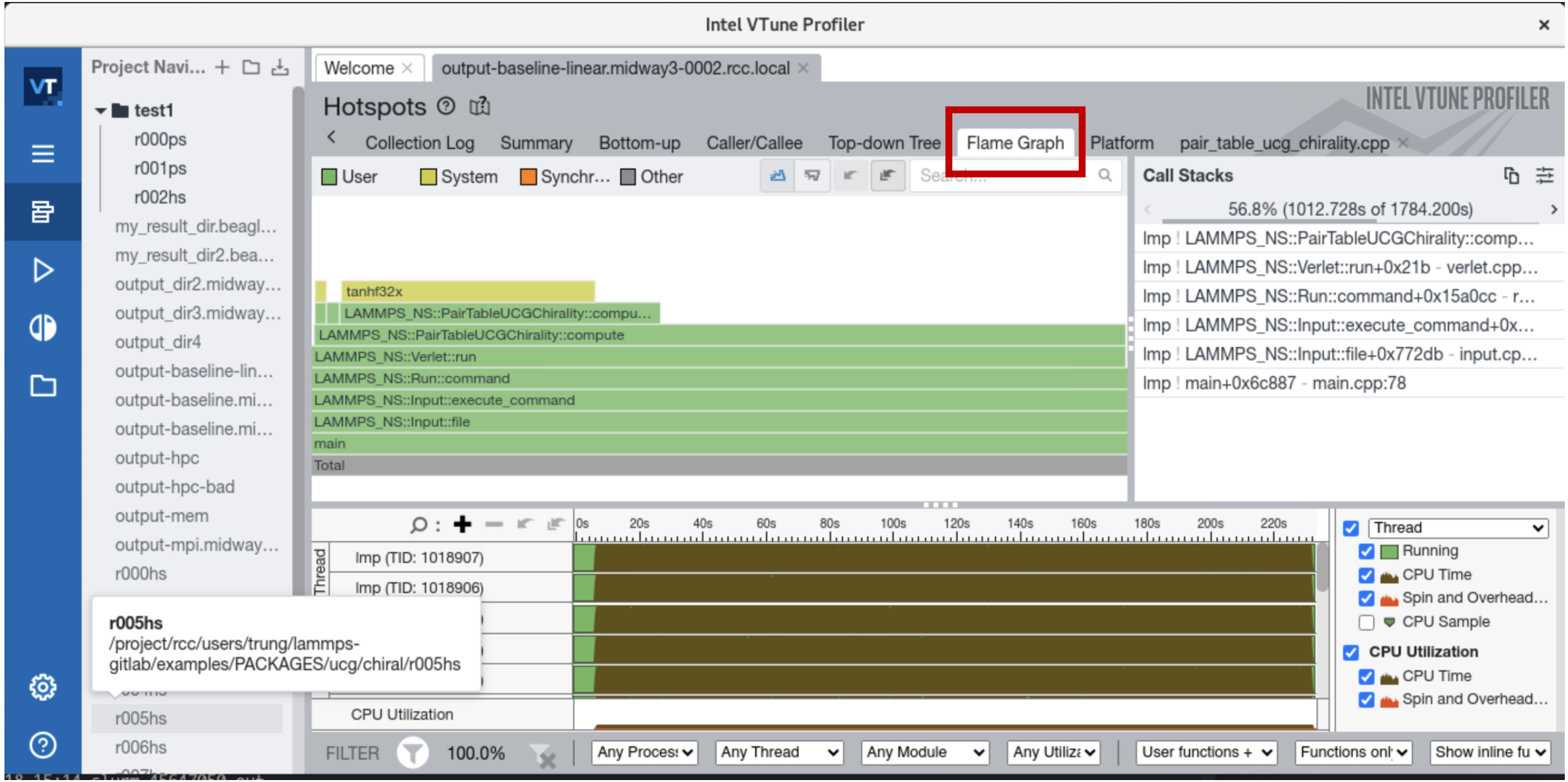
- vtune-gui ./output_dir



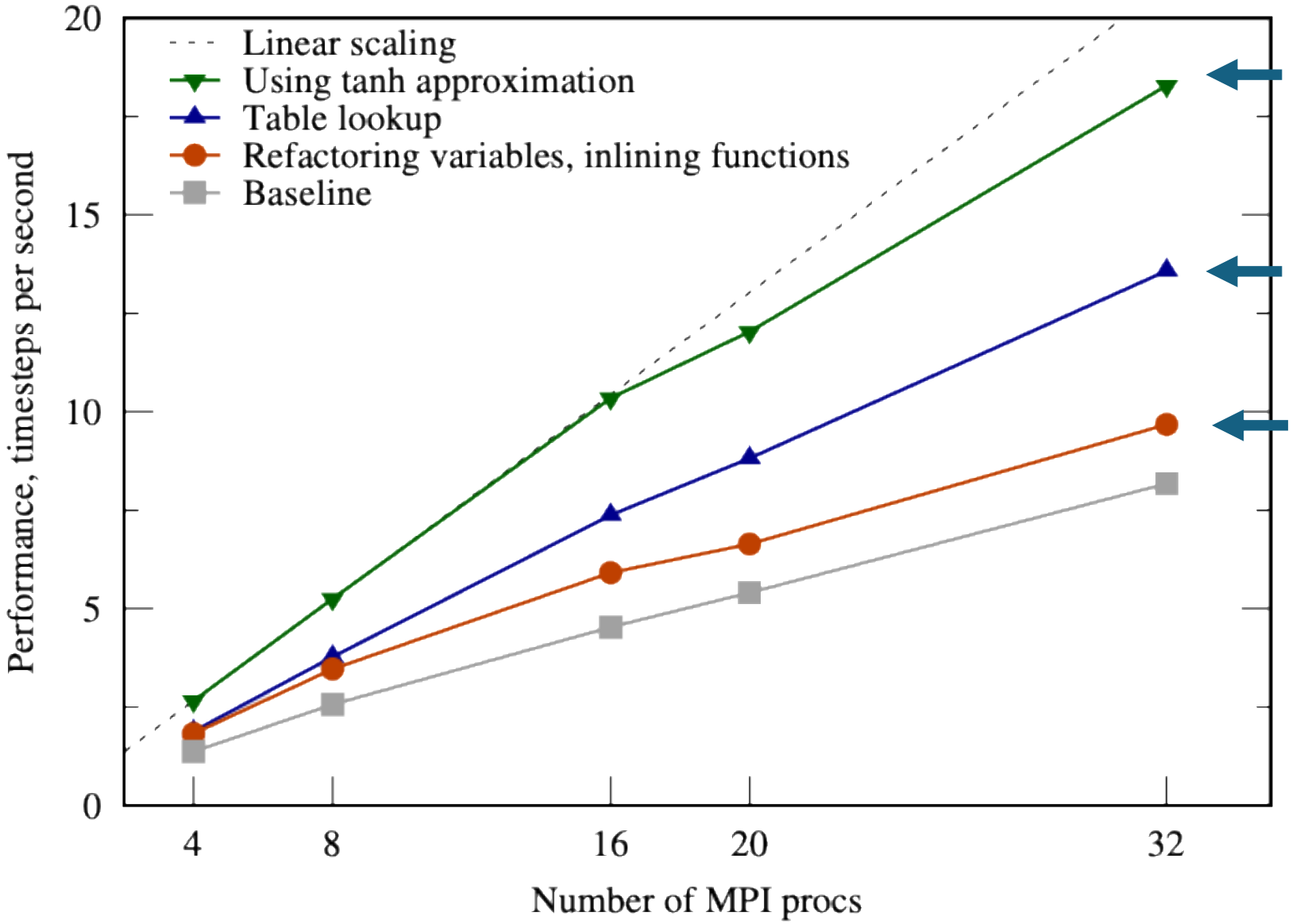

Intel VTune Profiler
Welcome
output-baseline-linear.midway3-0002.rcc.local
Hotspots
INTEL VTUNE PROFILER
Analysis Configuration
Collection Log
Summary
Bottom-up
Caller/Callee
Top-down Tree
Flame Graph
Platform
Grouping: Function / Call Stack
Function / Call Stack
CPU Time
LAMMPS_NS::PairTableUCGChirality::compute 1014.877s
LAMMPS_NS::Verlet::run ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS: 1012.728s
LAMMPS_NS::Verlet::setup ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_N 2.149s
tanhf32x 553.230s
LAMMPS_NS::PairTableUCGChirality::compute_proximity_function_der ← LAMMPS_NS::PairTableUCGChirality::comput 553.230s
LAMMPS_NS::Verlet::run ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_N 552.070s
LAMMPS_NS::Verlet::setup ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_ 1.160s
LAMMPS_NS::PairTableUCGChirality::compute_proximity_function_der 142.932s
LAMMPS_NS::PairTableUCGChirality::compute 142.932s
LAMMPS_NS::Verlet::run ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_N 142.652s
LAMMPS_NS::Verlet::setup ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_ 0.281s
LAMMPS_NS::Pair::ev_tally_full 28.399s
PMPI_Send 26.279s
LAMMPS_NS::NPairBin<(int)0, (int)1, (int)0, (int)0, (int)0>::build 7.470s
func@0x40a360 3.981s
PMPI_Allreduce 1.130s
Call Stacks
56.5% (80.740s of 142.932s)
lmp ! LAMMPS_NS::PairTableUCGChirality::compute_proximity_functi...
lmp ! LAMMPS_NS::PairTableUCGChirality::compute+0x63a900 - pair...
lmp ! LAMMPS_NS::Verlet::run+0x21b - verlet.cpp:316
lmp ! LAMMPS_NS::Run::command+0x15a0cc - run.cpp:176
lmp ! LAMMPS_NS::Input::execute_command+0x76bec - input.cpp:868
lmp ! LAMMPS_NS::Input::file+0x772db - input.cpp:313
lmp ! main+0x6c887 - main.cpp:78
0s 20s 40s 60s 80s 100s 120s 140s 160s 180s 200s 220s
Thread
lmp (TID: 1018907)
lmp (TID: 1018906)
lmp (TID: 1018914)
lmp (TID: 1018869)
lmp (TID: 1018896)
lmp (TID: 1018886)
lmp (TID: 1018891)
lmp (TID: 1018901)
Thread
Running
CPU Time
Spin and Overhead...
CPU Sample
CPU Utilization
CPU Time
Spin and Overhead...

Hotspots highlighted in the source code (-g)




Intel VTune Profiler
Project Navi...
test1
r000ps
r001ps
r002hs
my_result_dir.beagl...
my_result_dir2.bea...
output_dir2.midway...
output_dir3.midway...
output_dir4
output-baseline-lin...
output-baseline.mi...
output-baseline.mi...
output-hpc
output-hpc-bad
output-mem
output-mpi.midway...
r000hs
r005hs
r006hs
r005hs
/project/rcc/users/trung/lammps-gitlab/examples/PACKAGES/ucg/chiral/r005hs
Welcome
output-baseline-linear.midway3-0002.rcc.local
Hotspots
INTEL VTUNE PROFILER
Collection Log
Summary
Bottom-up
Caller/Callee
Top-down Tree
Flame Graph
Platform
pair_table_ucg_chirality.cpp
User
System
Synchr...
Other
tanhf32x
LAMMPS_NS::PairTableUCGChirality::compu...
LAMMPS_NS::PairTableUCGChirality::compute
LAMMPS_NS::Verlet::run
LAMMPS_NS::Run::command
LAMMPS_NS::Input::execute_command
LAMMPS_NS::Input::file
main
Total
Call Stacks
56.8% (1012.728s of 1784.200s)
lmp ! LAMMPS_NS::PairTableUCGChirality::comp...
lmp ! LAMMPS_NS::Verlet::run+0x21b - verlet.cpp...
lmp ! LAMMPS_NS::Run::command+0x15a0cc - r...
lmp ! LAMMPS_NS::Input::execute_command+0x...
lmp ! LAMMPS_NS::Input::file+0x772db - input.cp...
lmp ! main+0x6c887 - main.cpp:78
0s
20s
40s
60s
80s
100s
120s
140s
160s
180s
200s
220s
Thread
lmp (TID: 1018907)
lmp (TID: 1018906)
CPU Utilization
Thread
Running
CPU Time
Spin and Overhead...
CPU Sample
CPU Utilization
CPU Time
Spin and Overhead...
FILTER
100.0%
Any Process
Any Thread
Any Module
Any Utiliz
User functions +
Functions onl
Show inline fu

Optimizing codes step-by-step



tanh() is the bottleneck:
switch to an approximation

Table interpolation
scheme is the bottleneck:
linear → lookup

Cleanup the codes,
remove redundant
memory access,
make functions inline